

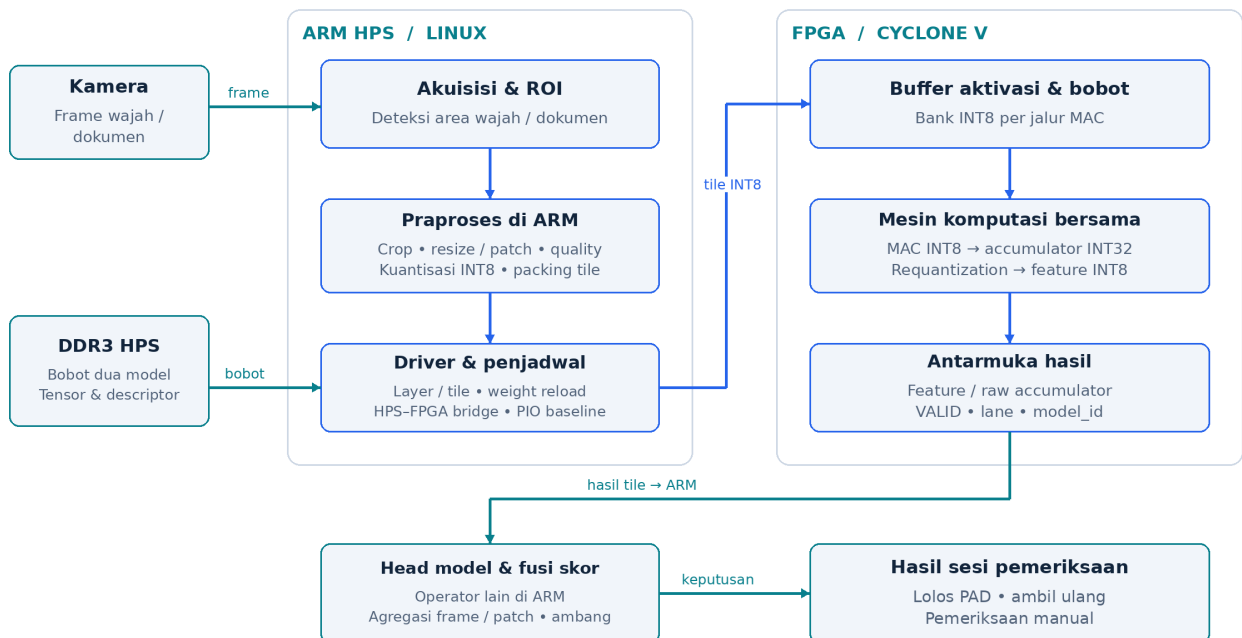
Diagram Blok Sistem dan Rincian Modul RTL

Accelerator CNN INT8 untuk deteksi serangan presentasi wajah dan dokumen e-passport.

QUANTPASS / DUALGUARD

Diagram Blok Sistem

Dua pemeriksaan PAD • satu datapath INT8 • model dijalankan bergantian



Baseline: PIO terimplementasi. DMA DDR3 & double buffering: pengembangan berikutnya.

Rancangan berbasis proposal tim • DE10-Nano / Cyclone V SoC • 7 Oktober 2026

Status: inti tile RTL dan CSR tersedia; simulasi fungsional lulus. Model terlatih, DMA, integrasi board dan hasil sintesis belum tersedia.

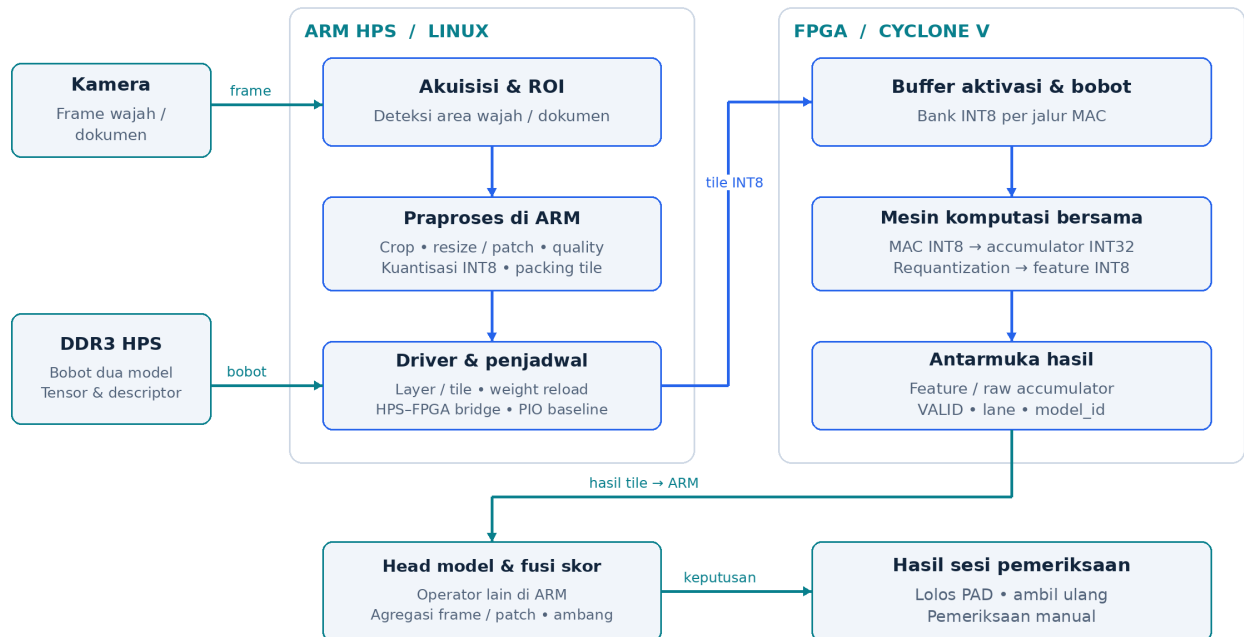
Paket memuat gambar siap pakai, penjelasan arsitektur, kontrak angka, peta register, prosedur driver dan seluruh kode RTL Verilog pada lampiran.

1. Diagram Blok Sistem

QUANTPASS / DUALGUARD

Diagram Blok Sistem

Dua pemeriksaan PAD • satu datapath INT8 • model dijalankan bergantian



Baseline: PIO terimplementasi. DMA DDR3 & double buffering: pengembangan berikutnya.

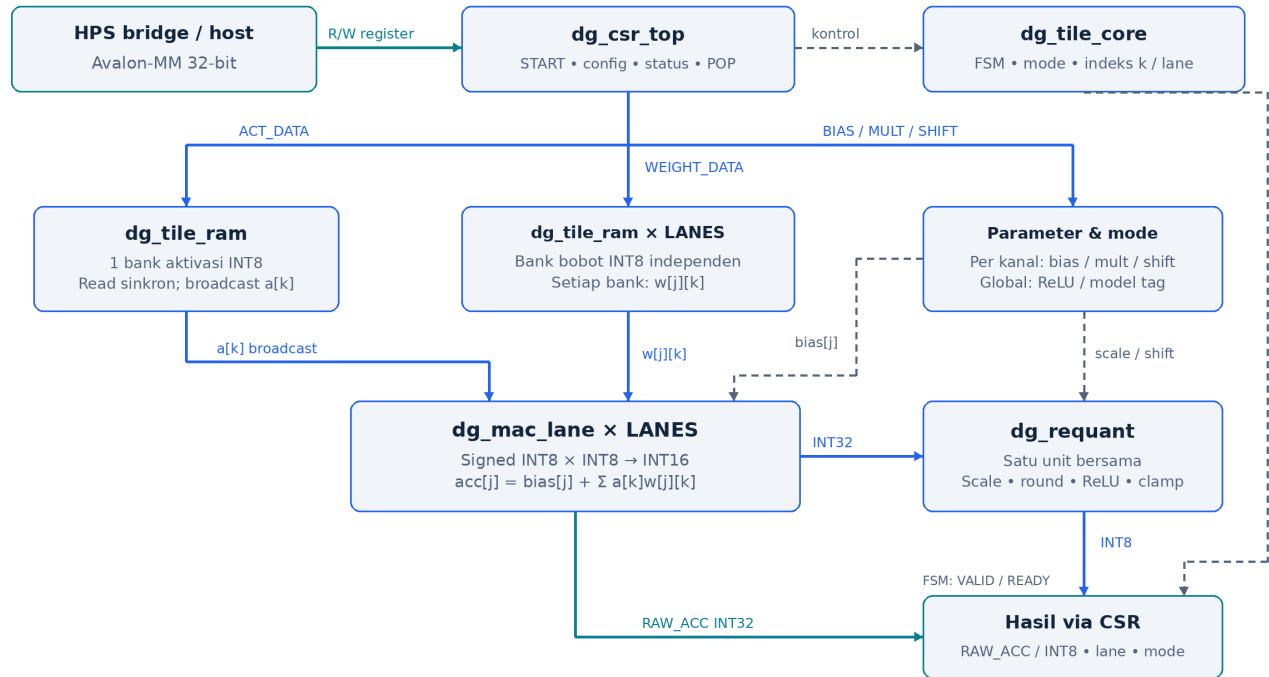
ARM memproses kamera dan mengatur model; FPGA menghitung tile. Feature keluaran tile dikembalikan ke ARM untuk menjalankan graph dan menghasilkan skor PAD. Bobot berasal dari DDR3 HPS melalui driver dan bridge, bukan sambungan langsung DDR3 ke MAC.

2. Arsitektur Internal RTL

QUANTPASS / DUALGUARD

Arsitektur Modul RTL

Sesuai hierarki Verilog • output-stationary • buffer dibagi per jalur



8 jalur default; 128 jalur target. Garis putus-putus: kontrol. DMA & scheduler graph berada di luar RTL ini.

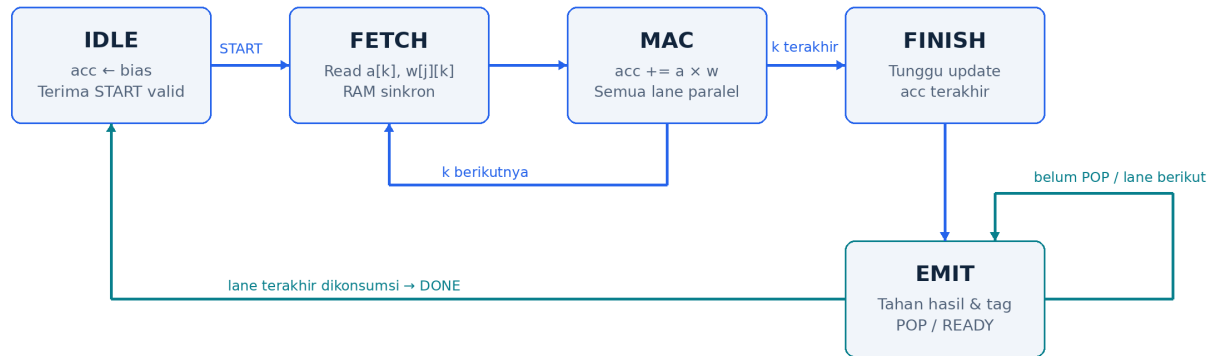
Setiap lane memiliki bank bobot dan accumulator sendiri; aktivasi disiarkan ke semua lane. Requantizer dipakai bersama. Kode memakai jalur PIO melalui CSR; DMA belum diimplementasikan.

3. Kendali dan Handshake

QUANTPASS / DUALGUARD

FSM Kendali Tile

Pembacaan RAM sinkron • accumulator diselesaikan sebelum hasil ditampilkan



Hasil stabil saat $VALID=1$ sampai POP diterima. FSM tidak mengizinkan perubahan konfigurasi ketika BUSY. DONE terjadi setelah semua hasil tile dikonsumsi, bukan hanya setelah operasi MAC selesai.

Rancangan prototipe riset mahasiswa untuk deteksi serangan presentasi (PAD) wajah dan dokumen. Disusun berdasarkan DualGuard_Analisis_v3.pdf serta tiga diagram referensi tim, 7 Oktober 2026. Bukan pembuktian keaslian kriptografis paspor atau pencocokan identitas.

Paket

- figures/Diagram_Blok_Sistem.png: pembagian kamera, ARM HPS, DDR3, FPGA, keputusan.
- figures/Arsitektur_Modul_RTL.png: modul dan jalur data yang sesuai dengan kode.
- figures/FSM_Tile.png: kendali tile dan backpressure.
- docs/DualGuard_Diagram_dan_RTL.pdf: uraian teknis, register, validasi dan lampiran seluruh RTL.
- rtl/: lima modul Verilog-2005, tidak memakai IP tertutup.
- tb/: testbench mandiri (testbench CSR memakai SystemVerilog).
- Makefile: kompilasi dan simulasi dengan Icarus Verilog.

3.1 Solusi dan Arsitektur Sistem

DualGuard mengimplementasikan satu mesin komputasi CNN INT8 yang digunakan bergantian untuk pemeriksaan wajah dan dokumen. ARM Cortex-A9 pada HPS mengelola akuisisi kamera, deteksi ROI, crop, resize atau patch, pemeriksaan kualitas, kuantisasi, penjadwalan layer/tile, serta agregasi skor antar-frame. Bobot model wajah dan dokumen berada di DDR3 HPS. Bobot tile terpilih disalin ke bank memori FPGA melalui jembatan HPS-FPGA. Penggantian model dilakukan dengan memuat bobot dan parameter baru setelah semua keluaran pekerjaan sebelumnya selesai dikonsumsi; bitstream datapath tetap.

Di FPGA, aktivasi INT8 dibaca dari buffer sinkron lalu disiarkan ke jalur MAC. Setiap jalur memiliki bank bobot sendiri, pengali signed 8x8 dan akumulator signed INT32. Mesin melakukan dot product untuk beberapa kanal keluaran dari satu posisi spasial. Requantizer bersama mengubah hasil INT32 menjadi INT8 secara berurutan. Hasil menyertakan nomor jalur dan identitas mode untuk mencegah pertukaran hasil antarmodel. ARM mengolah keluaran layer menjadi logits/skor melalui head model dan operator yang belum didukung; hasil tile bukan otomatis skor PAD.

Skor wajah dan dokumen dikalibrasi terpisah. Dalam sesi yang mewajibkan dua pemeriksaan, status lolos PAD hanya diterbitkan jika kedua cabang memenuhi ambang validasi dan kualitas gambar memadai. Skor meragukan atau kualitas buruk memicu ambil ulang, sedangkan hasil di bawah ambang memicu pemeriksaan manual. Jumlah frame, patch dan ambang ditetapkan pada validasi, bukan diubah berdasarkan data uji.

Batas implementasi yang harus disampaikan saat presentasi

Kode ini adalah accelerator tile konvolusi/dot-product dan register kontrol, bukan CNN lengkap siap membaca kamera. Kamera, Linux driver, graph runtime, model terlatih, DMA DDR3, clock/reset board, pin assignment, Platform Designer dan Quartus project belum disertakan. Baseline memakai programmed I/O (PIO); DMA dan double buffering adalah pengembangan untuk menekan waktu transfer. Tidak ada klaim latensi, akurasi, daya atau penggunaan DSP hasil board.

MiniFASNetV2 wajah 80x80 dan MobileNetV2 width 0.5 dokumen atau CNN patch 64x64 adalah kandidat proposal, belum merupakan model yang terintegrasi. Audit graph aktual wajib dilakukan: depthwise convolution, PReLU, residual add, pooling, SE, FC dan softmax tidak otomatis tersedia dalam RTL ini. ARM menangani operator tersebut pada baseline. Depthwise dapat dipetakan satu jalur per pekerjaan dengan packing ulang, tetapi utilitasnya rendah; engine depthwise khusus diperlukan bila bottleneck besar.

Kontrak komputasi

Untuk jalur j dan panjang K :

$$\text{acc}[j] = \text{bias}[j] + \sum(a[k] * w[j][k], k=0..K-1)$$
$$q[j] = \text{clamp_INT8}(\text{round_away_from_zero}(\text{acc}[j] * \text{multiplier}[j] / 2^{\text{shift}[j]}))$$

Jika ReLU aktif, nilai negatif dijepit ke 0 sebelum saturasi INT8. Kuantisasi baseline simetris dengan zero-point aktivasi, bobot dan keluaran semuanya 0. Multiplier adalah integer positif/nonnegatif INT32 (0..2147483647), shift 0..62, bias memakai skala accumulator. Multiplier dan shift per kanal mewakili rasio skala input*bobot/output. Ini kontrak fixed point khusus, bukan requantization TFLite bit-exact. Model wajib dievaluasi ulang dengan golden model integer ini. Spesifikasi TFLite mengizinkan zero-point aktivasi nonnol dan bobot per kanal; model semacam itu tidak dapat langsung dipakai tanpa konversi tervalidasi atau ekstensi RTL.

Bias harus berada dalam rentang $-2^{30}..2^{30}-1$ untuk DEPTH=256; setiap penjumlahan akhir harus muat INT32. Tidak ada deteksi overflow runtime. Untuk bobot/aktivasi signed INT8, batas absolut konservatif dot product tanpa bias adalah $K*16384$ (4,194,304 untuk $K=256$). RAM tidak direset: host wajib mengisi semua alamat aktif sebelum START dan tidak membaca hasil sebelum VALID.

LANES default 8, DEPTH default 256. Parameter statis: $1 \leq \text{LANES} \leq 128$ dan $1 \leq \text{DEPTH} \leq 256$ pada rancangan ini. Target 128 jalur dalam proposal memerlukan synthesis/fitter terpisah. Pada kode ini, throughput rata-rata fase hitung tidak mencapai 128 MAC/cycle; FETCH dan MAC bergantian, sehingga fase hitung mengeluarkan LANES MAC per 2 clock.

Pemetaan konvolusi

ARM membentuk $a[k]$ dari jendela input untuk satu (y,x) , dengan $k=((ky*Kw+kx)*Cin+ci)$, termasuk padding nol. Untuk output channel j , $w[j][k]$ memakai urutan yang sama. Setiap lane menghasilkan satu kanal output pada posisi itu. Pointwise 1x1 berarti $K=Cin$. Jika $Cout > LANES$, ulangi per kelompok kanal; lane yang tidak terpakai diisi bobot/bias nol dan hasilnya diabaikan. Bobot dapat dipakai ulang untuk posisi spasial berikutnya, hanya aktivasi diganti.

Jika $K > DEPTH$, bagi k menjadi segmen. Bias hanya disertakan di segmen pertama, bias segmen lain nol. ARM menjumlahkan keluaran RAW_ACC seluruh segmen dalam integer lebar, lalu menerapkan requantization satu kali setelah seluruh segmen. Jangan menjumlahkan hasil INT8 yang telah disaturasi. Versi sekarang tidak mempertahankan partial sum antartile di FPGA. Pengembangan scheduler + partial-sum SRAM perlu dilakukan untuk inferensi efisien.

Rincian Modul RTL

Modul	Fungsi	Data/antarmuka	Catatan implementasi
dg_csr_top	Register dan handshake HPS	Avalon-MM 32-bit	readLatency=0, waitrequest=0, byte address, full-word writes
dg_tile_core	FSM, parameter kanal, mode, serialisasi hasil	start/busy/done/error; valid/ready	Mode dikunci saat START; bukan pemilih bobot otomatis
dg_tile_ram	Buffer sinkron aktivasi/bobot	INT8, alamat 16-bit	Satu bank aktivasi + LANES bank bobot; M10K hint bukan jaminan inference
dg_mac_lane	Multiply-accumulate per kanal	signed INT8xINT8 -> INT16 -> INT32	Output-stationary; clear memuat bias
dg_requant	Pengali skala, pembulatan, ReLU, saturasi	INT32xINT32 -> INT64 -> INT8	Satu unit bersama; critical path perlu evaluasi timing

FSM: IDLE menerima START valid dan mengisi acc=bias. FETCH membaca RAM sinkron. MAC menambahkan produk ke seluruh jalur. Jika belum akhir k, kembali FETCH. FINISH memberi satu siklus setelah update accumulator terakhir. EMIT mempertahankan hasil dan tag hingga POP dari host; setelah lane terakhir dikonsumsi, DONE berdenyut dan status sticky diperbarui pada clock berikutnya. START saat busy menghasilkan error; perubahan konfigurasi ketika busy ditolak. Error tidak membatalkan komputasi aktif.

Tidak diperlukan adder tree antarlane karena setiap lane mengakumulasi kanal keluaran berbeda. Bila nanti paralelisme diarahkan pada dimensi k, barulah tree reduction dibutuhkan.

Register map (offset byte dari base address komponen)

Offset	Register	Akses / isi
0x00	CONTROL	W: bit0 START; bit1 clear status; bit2 POP hasil. Gunakan satu perintah per write.
0x04	STATUS	R: bit0 BUSY, bit1 VALID, bit2 DONE sticky, bit3 ERROR sticky
0x08	LENGTH	RW: K, 1..DEPTH
0x0C	CONFIG	RW: bit0 model_id (0 wajah, 1 dokumen), bit1 ReLU
0x10	INDEX	RW: indeks aktivasi/bobot, 0..DEPTH-1
0x14	LANE	RW: bank bobot/parameter, 0..LANES-1
0x18	ACT_DATA	W: signed INT8 pada bit7:0, alamat INDEX
0x1C	WEIGHT_DATA	W: signed INT8 pada bit7:0, alamat INDEX dan LANE
0x20	BIAS	W: signed INT32 untuk LANE
0x24	MULTIPLIER	W: integer $0..2^{31}-1$ untuk LANE
0x28	SHIFT	W: 0..62 untuk LANE
0x30	RESULT_Q	R: INT8 sign-extended, valid hanya ketika VALID=1
0x34	RESULT_ACC	R: INT32 sebelum requantization, valid ketika VALID=1
0x38	RESULT_META	R: lane pada bit15:0; model_id pada bit16
0x3C	CYCLES	R: jumlah clock BUSY, termasuk waktu tunggu host/POP
0x40	LANES	R: parameter hardware
0x44	DEPTH	R: parameter hardware

CSR harus didaftarkan sebagai agent Avalon-MM sederhana nonburst, addressUnits=SYMBOLS (byte), readLatency=0, dengan akses 32-bit selaras. Read dan write tidak dilakukan sekaligus oleh host. Base address disalin dari Platform Designer, tidak dihardcode. Semua port berada pada satu clock; reset rst_n sinkron aktif rendah harus disinkronkan sebelum masuk. Jembatan HPS-FPGA menangani domain clock menurut konfigurasi sistem. IRQ aktif ketika DONE atau ERROR sticky; CLEAR menghapusnya. Nilai konfigurasi invalid ditolak dan nilai lama dipertahankan; driver wajib berhenti jika ERROR dan memperbaiki transaksi sebelum START.

Urutan driver

1. Tunggu BUSY=0, clear status dengan CONTROL=2.
2. Konfirmasi identitas model, bentuk tensor dan checksum blob di software; mode hardware hanya tag.
3. Tulis LENGTH, CONFIG. Isi aktivasi via INDEX+ACT_DATA.

4. Untuk setiap LANE, isi bobot via INDEX+WEIGHT_DATA; isi BIAS, MULTIPLIER, SHIFT. Memori lama boleh digunakan hanya jika driver menjamin cache bobot masih sesuai.
5. Periksa ERROR; jika ada, hentikan perintah dan muat ulang data/config yang benar.
6. Tulis CONTROL=1. Poll VALID/ERROR dengan timeout.
7. Saat VALID: baca RESULT_META, RESULT_ACC dan RESULT_Q; verifikasi model/lane; tulis CONTROL=4 untuk POP. Ulangi hingga LANES hasil.
8. Poll DONE setelah POP terakhir. Periksa ERROR dan catat CYCLES.
9. Saat mode berganti, tunggu selesai, clear status dan reload seluruh bobot/parameter tile model baru. Satu bit CONFIG tidak mengganti model sendiri.

Anggaran dan waktu

Payload RAM = $DEPTH \times (LANES + 1)$ byte. Pada 8x256, payload 2304 byte; 128x256, payload 33024 byte. Alokasi fisik berbeda: bank kecil dapat menghabiskan satu blok M10K per bank (misalnya 129 blok untuk 128 lane + 1 aktivasi), sehingga persentase memori harus berasal dari fitter, bukan ukuran payload.

Dengan konsumsi satu output setiap clock, latensi dari START diterima sampai POP terakhir adalah $2 \times K + 1 + LANES$ clock (tidak termasuk pemuatan PIO dan delay host). Untuk $K=256$, $LANES=8$: 521 clock; pada clock asumsi 100 MHz setara 5.21 us, bukan waktu inferensi model. Peak fase MAC rata-rata $LANES \times fclk/2$; $LANES=128$ dan $fclk=100$ MHz memberi 6.4 GMAC/s sebelum transfer/output/idle. Perkiraan 5.12 GMAC/s pada proposal memakai asumsi datapath berbeda; jangan salin sebagai hasil RTL ini. Target 100 MHz belum diverifikasi. Multiplier requantization 32x32 dan variable shift dapat menjadi bottleneck timing; pipeline diperlukan jika timing gagal.

Pemakaian DSP tidak dapat diasumsikan tepat tiga pengali per blok dari operator * Verilog. DSP packing tergantung fitter, lebar dan struktur datapath. Hitung DSP untuk MAC, requantizer dan kontrol dari report Quartus. DMA, double buffering dan burst transfer baru boleh diberi angka setelah integrasi.

Verifikasi

Jalankan make test. Testbench CSR mengecek 12 tile dengan bobot di-reload, 48 dot product terhadap perhitungan independen, $K=1$ dan $K=256$, bilangan signed ekstrem, ReLU, saturasi, output yang ditahan tanpa POP, tag mode/lane, larangan perubahan konfigurasi saat busy, serta error panjang, shift, multiplier dan byteenable. Testbench requantization mengecek 10 vektor batas termasuk tie negatif dan produk INT64. Testbench CSR memakai 4 lane untuk mempercepat simulasi; top default 8 lane juga dikompilasi. Konfigurasi 8 dan 128 lane telah dielaborasi dan disimulasikan dengan satu dot product signed ekstrem pada setiap lane; kedua uji lulus. Ini tidak membuktikan kelayakan resource atau timing 128 lane. Ini pengujian fungsional, bukan coverage lengkap atau pembuktian formal.

Uji board berikutnya: elaborasi $LANES=8/64/128$; report DSP/ALM/register/M10K; Fmax/timing; uji golden tensor tiap layer; port DDR3; timeout/reset; latensi total kamera+ARM+transfer+FPGA+fusi; biaya switching; APCER/BPCER/ACER pada split per subjek/dokumen; quantization loss dibanding FP32. Jangan mengklaim model lengkap/keamanan/efisiensi sebelum pengukuran.

Referensi

[1] Dokumen internal tim: DualGuard_Analisis_v3.pdf dan diagram model/sistem/accelerator yang dilampirkan.

[2] Terasic, DE10-Nano documentation:

<https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=1046&PartNo=4>

[3] Intel, Avalon Interface Specifications:

https://cdrdv2-public.intel.com/667068/mnl_avalon_spec-683091-667068.pdf

[4] TensorFlow, 8-bit quantization specification: https://github.com/tensorflow/tensorflow/blob/master/tensorflow/lite/g3doc/performance/quantization_spec.md

Referensi [2]-[4] untuk integrasi dan format angka, bukan bukti kinerja atau akurasi rancangan ini. Kandidat model mengikuti proposal dan belum divalidasi ulang di paket ini.

Lampiran A. Seluruh Kode RTL Verilog

Lima berkas di bawah sama dengan sumber dalam folder rtl/. Testbench, Makefile dan log hasil simulasi tersedia di ZIP. RTL dikompilasi dalam mode Verilog-2005; hanya testbench CSR memakai SystemVerilog.

dg_csr_top.v

```
001 `timescale 1ns/1ps
002 // Avalon-MM 32-bit agent: byte addresses, readLatency=0, no bursts.
003 // Full-word aligned writes only. Single clock domain, synchronous reset.
004 module dg_csr_top #(parameter LANES=8, DEPTH=256)(
005     input clk, input rst_n, input [7:0] address,
006     input read, input write, input [3:0] byteenable,
007     input [31:0] writedata, output reg [31:0] readdata,
008     output waitrequest, output irq
009 );
010 localparam [15:0] LANES_VALUE=LANES, DEPTH_VALUE=DEPTH;
011 reg [15:0] length, index_reg, lane_reg;
012 reg model_id, relu, sticky_done, sticky_error;
013 reg [31:0] cycles;
014 wire busy, done, error, valid, model;
015 wire [15:0] out_lane;
016 wire signed [31:0] out_acc;
017 wire signed [7:0] out_q;
018 wire aligned = address[1:0]==0;
019 wire wr = write && aligned && byteenable==4'hf;
020 wire ctrl = wr && address==8'h00;
021 wire load = wr && !busy;
022 wire pop = ctrl && writedata[2];
023 assign waitrequest=0;
024 assign irq=sticky_done || sticky_error;
025 dg_tile_core #(LANES,DEPTH) core(
026     .clk(clk),.rst_n(rst_n),.start(ctrl && writedata[0]),
027     .length(length),.model_id(model_id),.relu(relu),
028     .act_we(load && address==8'h18),.act_addr(index_reg),
029     .act_data(writedata[7:0]),
030     .weight_we(load && address==8'h1c),.weight_lane(lane_reg),
031     .weight_addr(index_reg),.weight_data(writedata[7:0]),
032     .param_we(load && (address==8'h20 || address==8'h24 || address==8'h28)),
033     .param_lane(lane_reg),
034     .param_kind(address==8'h20 ? 2'd0 : address==8'h24 ? 2'd1 : 2'd2),
035     .param_data(writedata),.busy(busy),.load_ready(),
036     .done(done),.error(error),.out_valid(valid),.out_ready(pop),
037     .out_lane(out_lane),.out_model(model),.out_acc(out_acc),.out_q(out_q));
038 always @* begin
039     readdata=0;
040     if(read && aligned) case(address)
```

dg_csr_top.v (lanjutan)

```
041      8'h04: readdata={28'd0,sticky_error,sticky_done,valid,busy};
042      8'h08: readdata={16'd0,length};
043      8'h0c: readdata={30'd0,relu,model_id};
044      8'h10: readdata={16'd0,index_reg};
045      8'h14: readdata={16'd0,lane_reg};
046      8'h30: readdata={{24{out_q[7]}},out_q};
047      8'h34: readdata=out_acc;
048      8'h38: readdata={15'd0,model,out_lane};
049      8'h3c: readdata=cycles;
050      8'h40: readdata={16'd0,LANES_VALUE};
051      8'h44: readdata={16'd0,DEPTH_VALUE};
052      default: readdata=0;
053  endcase
054 end
055 always @(posedge clk) begin
056   if(!rst_n) begin
057     length<=1; index_reg<=0; lane_reg<=0; model_id<=0; relu<=0;
058     sticky_done<=0; sticky_error<=0; cycles<=0;
059   end else begin
060     if(ctrl && writedata[1]) begin sticky_done<=0; sticky_error<=0; end
061     if(ctrl && writedata[0] && !busy) begin cycles<=0; sticky_done<=0; end
062     else if(busy) cycles<=cycles+1;
063     if(done) sticky_done<=1;
064     if(error) sticky_error<=1;
065     if(write && (!aligned || byteenable!=4'hf)) sticky_error<=1;
066     if(wr && !ctrl && busy) sticky_error<=1;
067     if(load) case(address)
068       8'h08: if(writedata<=DEPTH && writedata>0) length<=writedata[15:0];
069       else sticky_error<=1;
070       8'h0c: if(writedata<=3) begin model_id<=writedata[0]; relu<=writedata[1]; end
071       else sticky_error<=1;
072       8'h10: if(writedata<DEPTH) index_reg<=writedata[15:0]; else sticky_error<=1;
073       8'h14: if(writedata<LANES) lane_reg<=writedata[15:0]; else sticky_error<=1;
074       8'h00,8'h18,8'h1c,8'h20,8'h24,8'h28: ;
075       default: sticky_error<=1;
076     endcase
077   end
078 end
079 endmodule
```

dg_mac_lane.v

```
001 `timescale 1ns/1ps
002 module dg_mac_lane(
003   input clk, input rst_n, input clear, input enable,
004   input signed [7:0] a, input signed [7:0] w,
005   input signed [31:0] bias, output reg signed [31:0] acc
006 );
007   wire signed [15:0] product = a * w;
008   wire signed [31:0] extended_product = {{16{product[15]}},product};
009   always @(posedge clk) begin
010     if (!rst_n) acc <= 0;
011     else if (clear) acc <= bias;
012     else if (enable) acc <= acc + extended_product;
013   end
014 endmodule
```

dg_requant.v

```
001 `timescale 1ns/1ps
002 // Custom symmetric INT8 contract; NOT TFLite bit-exact requantization.
003 // multiplier must be 0..2^31-1, shift 0..62. Round ties away from zero.
004 module dg_requant(
005     input signed [31:0] acc, input signed [31:0] multiplier,
006     input [5:0] shift, input relu, output reg signed [7:0] value
007 );
008     reg signed [63:0] product, rounded;
009     reg [63:0] magnitude, half, scaled;
010     always @* begin
011         product = acc * multiplier;
012         magnitude = product[63] ? -product : product;
013         half = (shift == 0) ? 64'd0 : (64'd1 << (shift-1));
014         scaled = (magnitude + half) >> shift;
015         rounded = product[63] ? -$signed(scaled) : $signed(scaled);
016         if (relu && rounded < 0) value = 0;
017         else if (rounded > 127) value = 8'sd127;
018         else if (rounded < -128) value = -8'sd128;
019         else value = rounded[7:0];
020     end
021 endmodule
```

dg_tile_core.v

```
001 `timescale 1ns/1ps
002 // Output-stationary tile: one activation vector, LANES output channels.
003 // Host packs convolution windows and schedules tiles. No DMA in this module.
004 module dg_tile_core #(parameter LANES=8, DEPTH=256)(
005     input clk, input rst_n,
006     input start, input [15:0] length, input model_id, input relu,
007     input act_we, input [15:0] act_addr, input signed [7:0] act_data,
008     input weight_we, input [15:0] weight_lane, input [15:0] weight_addr,
009     input signed [7:0] weight_data,
010     input param_we, input [15:0] param_lane, input [1:0] param_kind,
011     input [31:0] param_data,
012     output busy, output load_ready, output reg done, output reg error,
013     output out_valid, input out_ready, output [15:0] out_lane,
014     output out_model, output signed [31:0] out_acc,
015     output signed [7:0] out_q
016 );
017     localparam IDLE=0, FETCH=1, MAC=2, FINISH=3, EMIT=4;
018     reg [2:0] state;
019     reg [15:0] k, count, lane;
020     reg active_model, active_relu;
021     reg signed [31:0] biases [0:LANES-1];
022     reg signed [31:0] multipliers [0:LANES-1];
023     reg [5:0] shifts [0:LANES-1];
024     wire signed [7:0] a;
025     wire signed [31:0] accumulators [0:LANES-1];
026     wire accept_start = start && state==IDLE && length>0 && length<=DEPTH;
027     assign busy = state != IDLE;
028     assign load_ready = !busy;
029     assign out_valid = state == EMIT;
030     assign out_lane = lane;
031     assign out_model = active_model;
032     assign out_acc = accumulators[lane];
033     dg_tile_ram #(DEPTH) input_bank(clk, act_we && !busy,
034         act_addr, act_data, state==FETCH, k, a);
035     genvar g;
036     generate for(g=0; g<LANES; g=g+1) begin: banks
037         wire signed [7:0] w;
038         dg_tile_ram #(DEPTH) weight_bank(clk,
039             weight_we && !busy && weight_lane==g,
040             weight_addr, weight_data, state==FETCH, k, w);
```

dg_tile_core.v (lanjutan)

```
041     dg_mac_lane mac(clk,rst_n,accept_start,state==MAC,a,w,  
042         biases[g],accumulators[g]);  
043 end endgenerate  
044 // One shared requantizer selected by output lane.  
045 dg_requant quant(out_acc,multipliers[lane],shifts[lane],active_relu,out_q);  
046 integer i;  
047 always @(posedge clk) begin  
048     if(!rst_n) begin  
049         state<=IDLE; k<=0; count<=0; lane<=0;  
050         active_model<=0; active_relu<=0; done<=0; error<=0;  
051         for(i=0;i<LANES;i=i+1) begin  
052             biases[i]<=0; multipliers[i]<=1; shifts[i]<=0;  
053         end  
054     end else begin  
055         done<=0; error<=0;  
056         if(param_we && !busy && param_lane<LANES) begin  
057             case(param_kind)  
058                 0: biases[param_lane]<=param_data;  
059                 1: if(!param_data[31]) multipliers[param_lane]<=param_data;  
060                     else error<=1;  
061                 2: if(param_data<=62) shifts[param_lane]<=param_data[5:0];  
062                     else error<=1;  
063                 default: error<=1;  
064             endcase  
065         end  
066         if(start && busy) error<=1;  
067         case(state)  
068             IDLE: if(start) begin  
069                 if(length==0 || length>DEPTH) error<=1;  
070                 else begin  
071                     count<=length; k<=0; lane<=0;  
072                     active_model<=model_id; active_relu<=relu; state<=FETCH;  
073                 end  
074             end  
075             FETCH: state<=MAC;  
076             MAC: if(k==count-1) state<=FINISH;  
077                 else begin k<=k+1; state<=FETCH; end  
078             FINISH: state<=EMIT;  
079             EMIT: if(out_ready) begin  
080                 if(lane==LANES-1) begin state<=IDLE; done<=1; end
```

dg_tile_core.v (lanjutan)

```
081         else lane<=lane+1;  
082     end  
083     default: state<=IDLE;  
084 endcase  
085 end  
086 end  
087 endmodule
```

dg_tile_ram.v

```
001 `timescale 1ns/1ps  
002 // One synchronous-read bank. Contents are intentionally not reset.  
003 module dg_tile_ram #(parameter DEPTH=256)(  
004     input clk, input we, input [15:0] waddr, input signed [7:0] wdata,  
005     input re, input [15:0] raddr, output reg signed [7:0] q  
006 );  
007 (* ramstyle = "M10K" *) reg signed [7:0] mem [0:DEPTH-1];  
008 always @(posedge clk) begin  
009     if (we && waddr < DEPTH) mem[waddr] <= wdata;  
010     if (re && raddr < DEPTH) q <= mem[raddr];  
011 end  
012 endmodule
```